Middle Node

AlgoExpert · Easy · Linked Lists

The problem

- Return the middle node (second middle for even lengths) via slow/fast pointers.

Intuition

- A fast pointer moving two steps lands at the end exactly when a slow pointer moving one step reaches the middle.

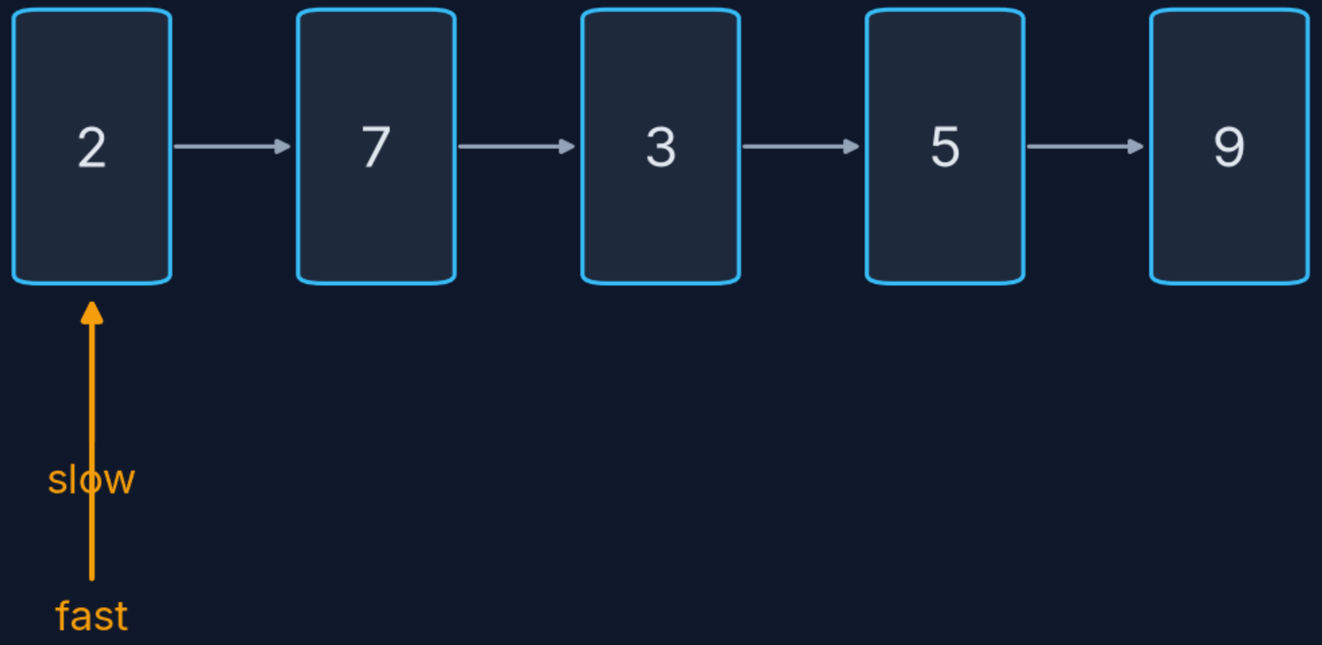
Brute force vs optimal

- Two passes: count length, then walk n/2 → O(n)
- Or copy into an array → O(n) space
- One pass: slow/fast pointers → O(n), O(1) space

Approach

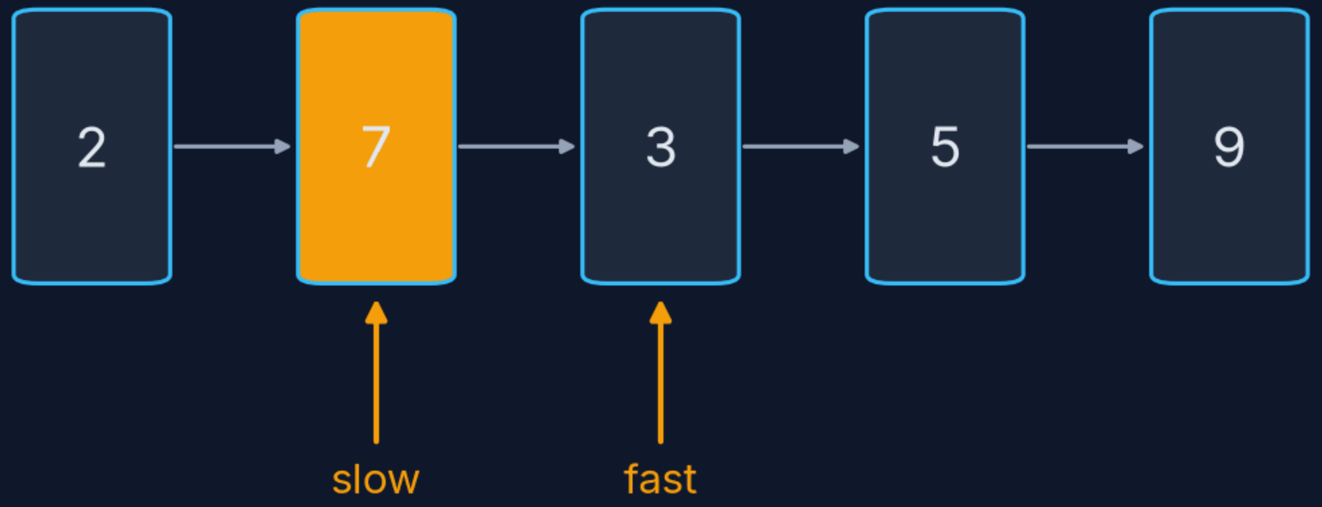
- 1. Start slow and fast at the head
- 2. Advance slow by one, fast by two
- 3. Stop when fast runs out
- 4. Return slow

Worked example



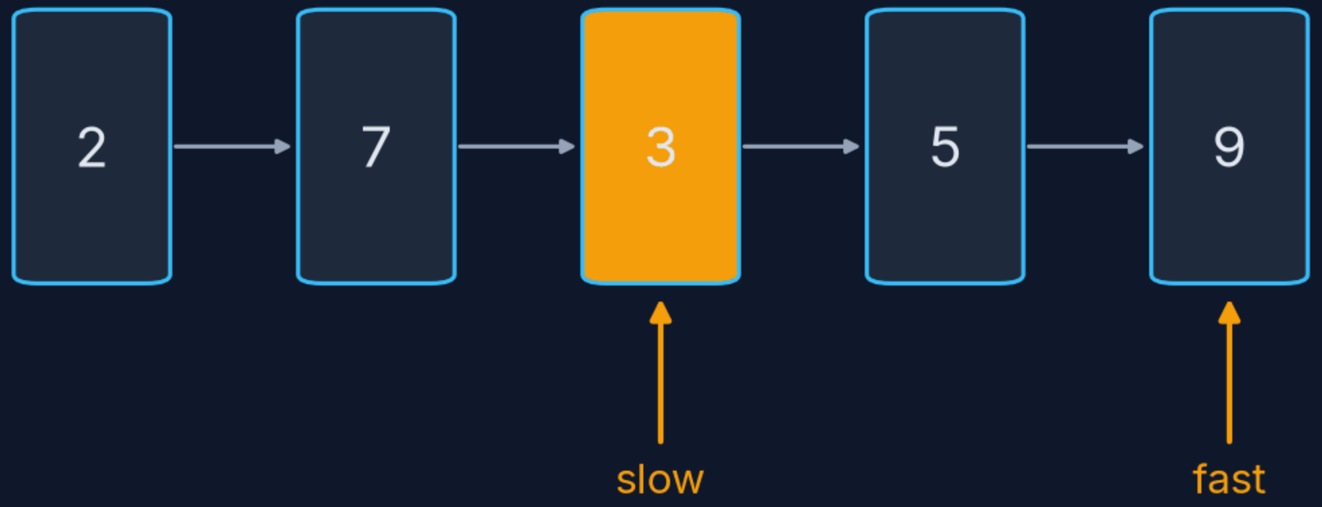
slow and fast start at the head

Worked example



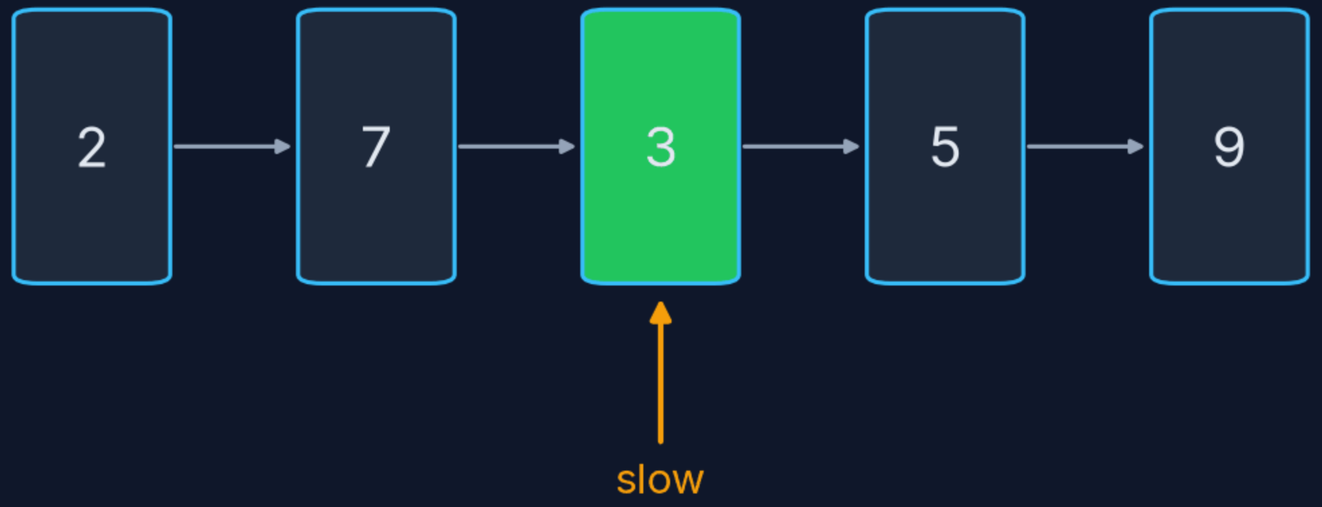
slow → 7, fast → 3

Worked example



slow → 3, fast → 9

Worked example



fast can't move: middle is 3

Java solution

```java
public class MiddleNode {
    public static LinkedListNode middleNode(LinkedListNode head) {
        LinkedListNode slow = head, fast = head;
        while (fast != null && fast.next != null) { slow = slow.next; fast = fast.next.next; }
        return slow;
    }
}
```

Complexity

- Time: O(n)
- Space: O(1)

Edge cases & pitfalls

- Even length: return the SECOND middle
- Single node → itself
- Check fast and fast.next before moving

Interview tips & follow-ups

- The same trick detects cycles (Floyd)
- Follow-up: split the list in half for merge sort

Recap

- A fast pointer moving two steps lands at the end exactly when a slow pointer moving one step reaches the middle.
- Complexity: O(n)